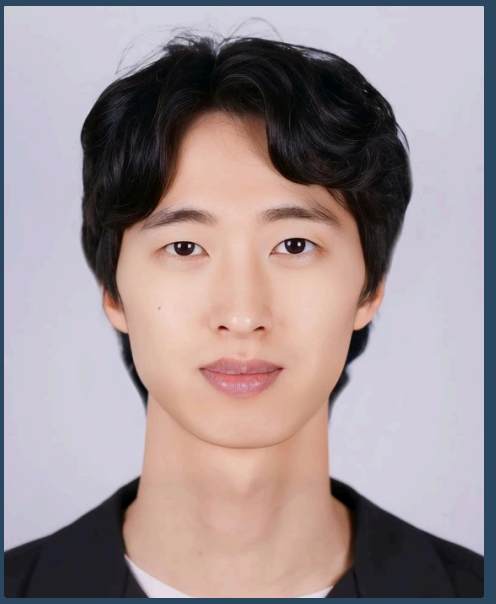
로봇 제어 · 임베디드 신입

정승호 SEUNGHO JUNG

펌웨어부터 로봇팔 제어까지
어느 단계도 가볍게 넘기지 않습니다



프로젝트와 실습에서 사용한 기술

01 로봇·비전

Python · ROS2 · OpenCV

YOLO · TFLite

02 제어·임베디드

C · C++ · STM32

FreeRTOS · PLC

03 설계·개발 도구

SolidWorks · Ansys · MATLAB

MuJoCo · Isaac Sim

학력과 교육, 자격 사항

기계공학 전공 지식을 바탕으로 로봇 제작과 제어를 학습했습니다

학력·교육

수료 | 2026.04 – 2026.11

대한상공회의소 서울기술교육센터

AI융합 로봇 SW개발자 3기

Python·Linux AI 프로그래밍
ROS/ROS2 통신·제어
Cortex-M·PLC·산업용 네트워크

졸업 | 2019.03 – 2026.02

순천향대학교 기계공학과

학점 3.65 / 4.5

졸업 | 2016.03 – 2019.02

대성고등학교

자격·어학

TOEIC 705점

2025.12

운전면허 1종 보통

2019.08 · 경찰청

초경량비행장치조종자

2019.09 · 한국교통안전공단

지그 위치에 동작을 맞추고 부품 안착까지 검사하는 로봇팔

ROS2 요청을 받아 지그 위치를 다시 재고, 저장한 동작을 그 위치에 맞춰 실행했습니다

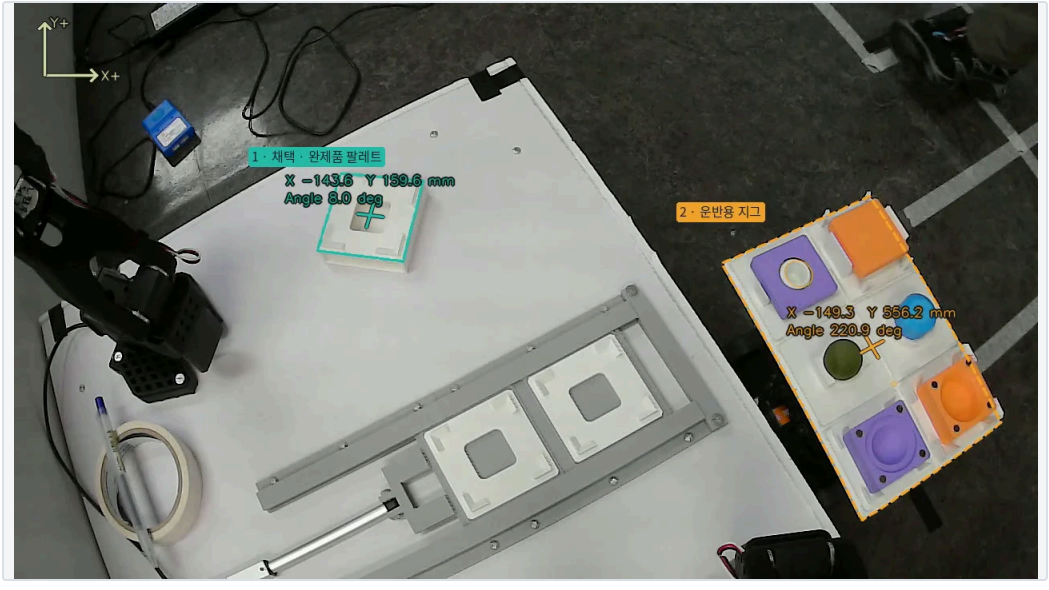


그림 1. 지그 이동 후 위치·방향을 다시 채택하는 화면

프로젝트 개요

로봇팔 2대가 지그 도착부터 조립, 완제품 하차까지 맡는 작업대를 만들었습니다
정차 위치가 달라지고 부품이 서로 가려도 공정이 이어지게 했습니다

핵심 기능

카메라로 **지그를 다시 측정해 동작 목표를 보정**하고,
부품 종류와 안착 상태를 검사했습니다

제어 방식

ROS2 Action으로 작업 요청을 받고,
순기구학·역기구학으로 집게 목표와 관절값을 계산했습니다

Python · OpenCV | YOLO · CAD 정합 | FK · IK | ROS2 Action | MuJoCo · Isaac Sim

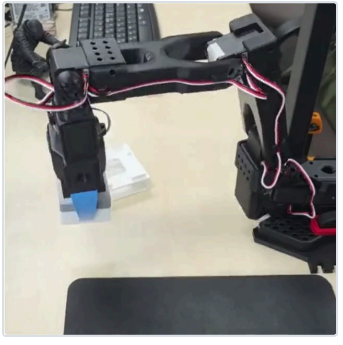
기간	공정	장비	인식 대상
2026.09 – 2026.11	지그 도착 → 조립 → 완제품 하차	SO-101 2대 · Raspberry Pi5	부품·지그 13종

작업 요청부터 관절 목표까지 연결

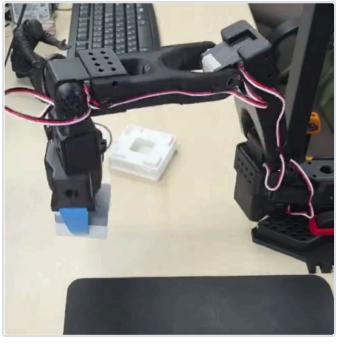
ROS2 요청을 받아 지그를 측정하고, 지그의 이동·회전만큼 집게 목표와 관절값을 다시 계산했습니다



처음 위치



지그 이동 후



다시 이동·회전 후

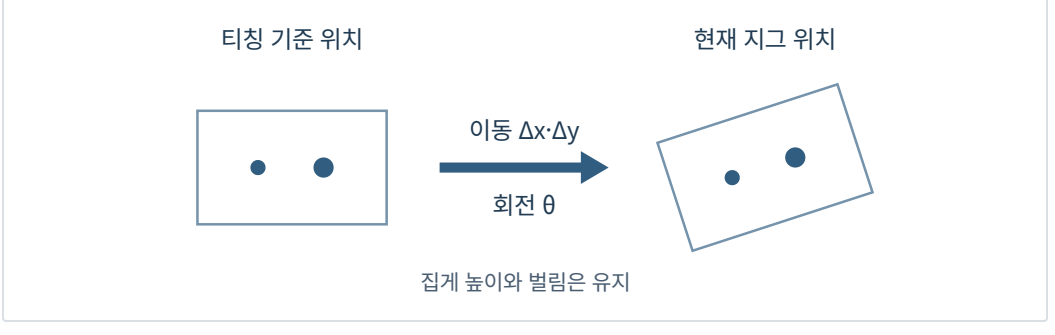


그림 2. 지그를 옮기고 돌려도 저장한 동작으로 부품을 올려놓는 시연(위)과 목표 보정 방식(아래)

ROS2 요청과 결과 반환

조립·하차 요청을 받아 지그 측정·시작 검사 후 실행

진행·완료·실패 상태와 검사·오류 기록 반환

순기구학과 목표 변환

티칭 관절값과 TCP로 집게 자세 계산

지그 이동·회전만큼 집게 목표 보정

역기구학과 도착 확인

관절 범위·자세 연속성 확인 후 관절값 계산

모터 실측 각도로 목표 도착 확인

AI 인식과 설계 형상 맞춤으로 지그·팔레트 위치 확정

외곽선만으로는 가려진 팔레트를 놓쳐, AI가 찾은 영역을 설계 형상에 다시 맞추고 위치·방향이 맞을 때만 채택했습니다



그림 3. 운반용 지그(위)와 완제품 팔레트(아래)의 위치를 찾는 과정

운반용 지그 · 360° 방향 검증

AI 후보 영역을 외곽·격자·원형 특징으로 검증

첫 3관측이 **0.5mm·0.5°** 이내면 바로 채택

아니면 4초 판단 후 과반 그룹 중앙값, 5회 실패 시 중단

완제품 팔레트 · CAD 정합

AI가 찾은 영역에 설계 형상을 맞춰

가려진 부분까지 위치 계산

285장 중 284장에서 팔레트를 검출

팔레트 검출률	외곽선 검출	AI 인식 + 외곽선	AI 인식 + 설계 형상 맞춤 · 채택
촬영 사진 285장 · 최초 검출 시	**79.3%** (226/285장)	**80.7%** (230/285장)	**99.6%** (284/285장)

티칭 GUI와 Windows 관제 화면

동작을 가르치는 화면과 작업을 요청하는 화면을 만들고, 통신이 끊기면 팔이 멈추게 했습니다

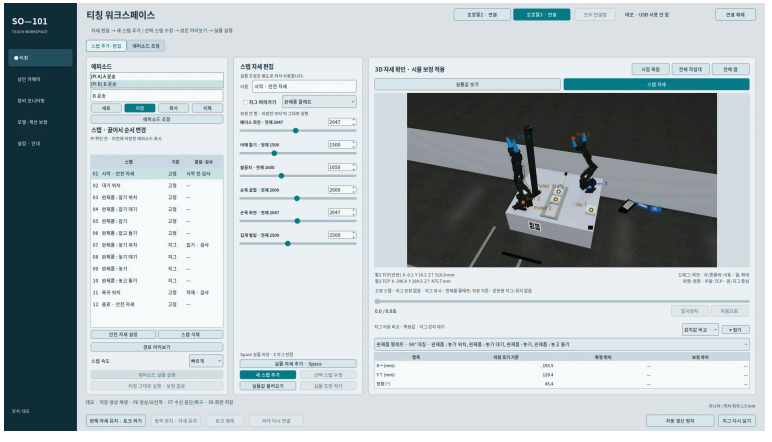


그림 4. 티칭 GUI · 스텝 편집·카메라 측정·검사

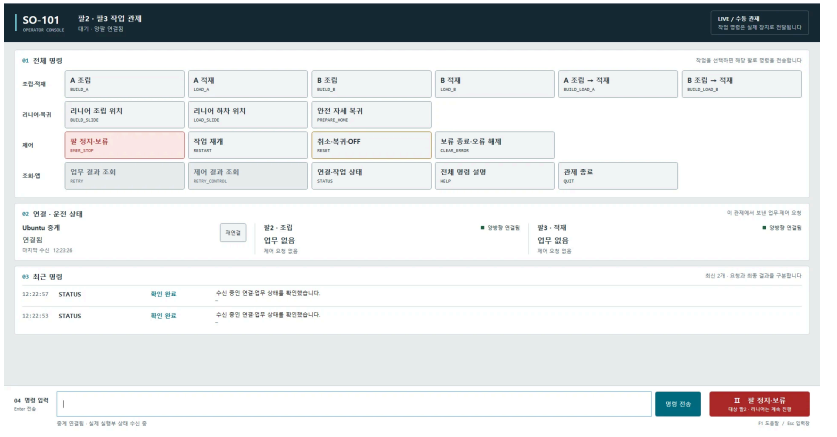


그림 5. Windows 관제 화면 · 명령 요청과 연결·운전 상태

티칭 GUI · Python Tkinter

기준 설정과 스텝 편집을 한 화면에서 처리

지그 측정과 **안착 검사 결과를 함께 표시**

관제 화면 · Windows WPF

조립·적재·정지·재개 명령 요청과 결과 조회

Ubuntu 중계로 **ROS2 Action 클라이언트와 연결**

통신 단절과 정지 처리

0.25초마다 생존 신호 교환

3초간 응답이 없으면 팔 정지·작업 보류

스마트폰으로 조작하는 이동형 로봇

4륜 이동체와 로봇팔을 결합해 스마트폰으로 주행과 물품 회수 동작을 제어했습니다

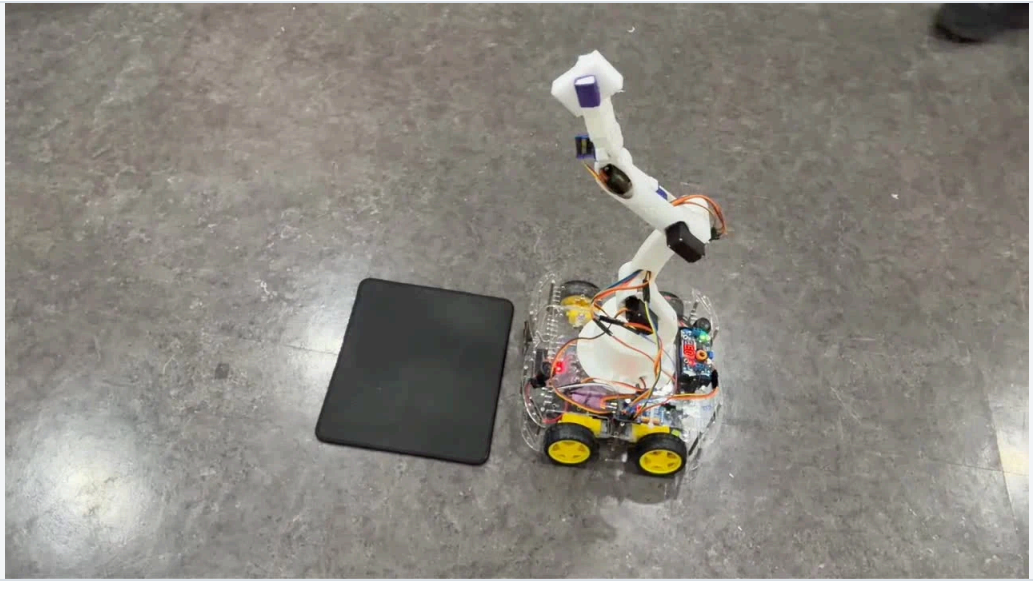


그림 6. 원격 물품 회수·운반 로봇 시연품

프로젝트 개요

이동체와 로봇팔을 결합한 물품 회수·운반 시연품을 만들었습니다
앱으로 주행과 관절을 수동 조작하고, 저장한 동작을 재생하게 했습니다

펌웨어·통신

STM32F411과 FreeRTOS로 펌웨어를 작성하고,
앱과는 Bluetooth로 **16바이트 명령·응답**을 주고받았습니다

구동·저장

서보 6개는 PCA9685로 제어하고, 티칭한 동작은 Flash에 저장했습니다

STM32F411 | C · FreeRTOS | Flutter | Bluetooth

수행 시기	제어 보드	구동 구성	구현 범위
2026.07 · 1주	**STM32F411**	**4WD · 서보 6개**	**수동 조작 · 티칭**

16바이트 통신 규격과 구동 문제 개선

16바이트 규격으로 앱과 펌웨어를 연결하고, 부하 시 떨림과 급격한 움직임을 줄였습니다

16바이트 명령·응답

주행·로봇팔·티칭·설정을 모드로 구분
앱 요청과 펌웨어 응답을 연결해 처리 상태 확인

서보 제어와 동작 저장

PCA9685로 서보 6개 제어
Flash에 티칭·설정 저장, 최대 **12개 시퀀스 × 30개 웨이포인트**

동작 간 간섭 방지

로봇팔 이동·Flash 작업 중 주행 잠금
500ms 동안 유효한 주행 명령이 없으면 이동체 정지

문제	원인	개선
부하가 걸릴 때의 떨림	서보 단자 전압이 순간적으로 낮아짐	전원 배선 점검, 커패시터 추가, 서보 교체로 **떨림 감소**
관절의 급격한 움직임	펄스폭이 한 번에 바뀌어 속도가 불연속	**20ms마다 펄스폭을 단계적으로 바꾸는 S-curve** 적용
관절 원점·가동 범위	필요한 펄스값이 서보마다 다름	**−90°·0°·+90° 3점 보정**으로 원점·리미트 설정
고속 주행 시 방향 보정	PWM이 최댓값에 닿아 PID 보정이 반영되지 않음	**최대 출력을 220 PWM으로 제한**

소재 공급부터 창고 적재까지의 공정 제어

MPS 트레이너에서 소재 분류와 1–4층 창고 적재·비움 동작을 구현했습니다

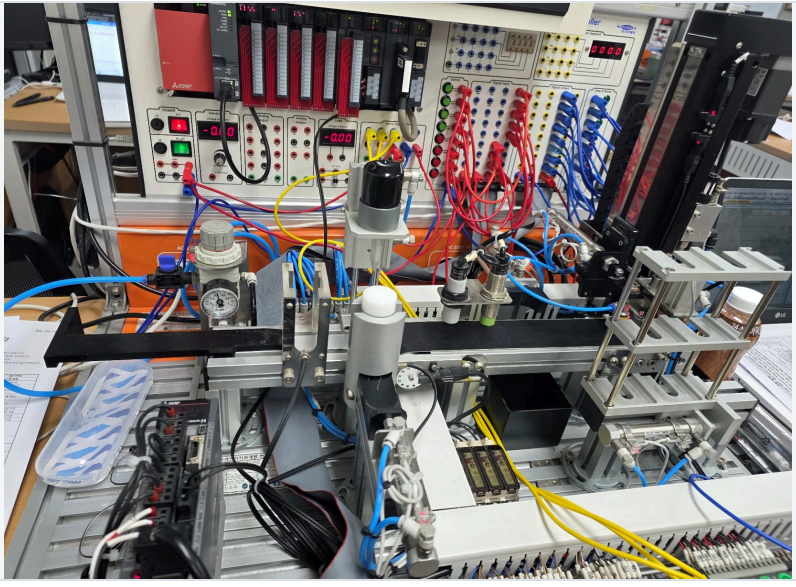


그림 7. MPS 트레이너 설비

공정 구성

컨베이어·실린더·서보모터로 공급·가공·분류·저장 공정을 연결했습니다

병렬 적재

금속·비금속을 판별해 **1~4층 창고에 병렬로 적재**하고,
수동 모드에서는 층별로 개별 제어할 수 있게 했습니다

HMI 구성

HMI 화면에서 가공 횟수와 시간을 지정하고 적재 상태를 확인하게 했습니다

Mitsubishi Q PLC | GX Works2 | GT Designer3

수행 시기	PLC	래더 프로그램	HMI 작화
2026.06 · 3일	**Mitsubishi Q 시리즈**	**GX Works2**	**GT Designer3**

병렬 시퀀스의 상태 관리와 코드 통합

저장 가능 여부를 판단하는 메모리와 실제 적재 정보를 분리해 시퀀스 간 간섭을 줄였습니다

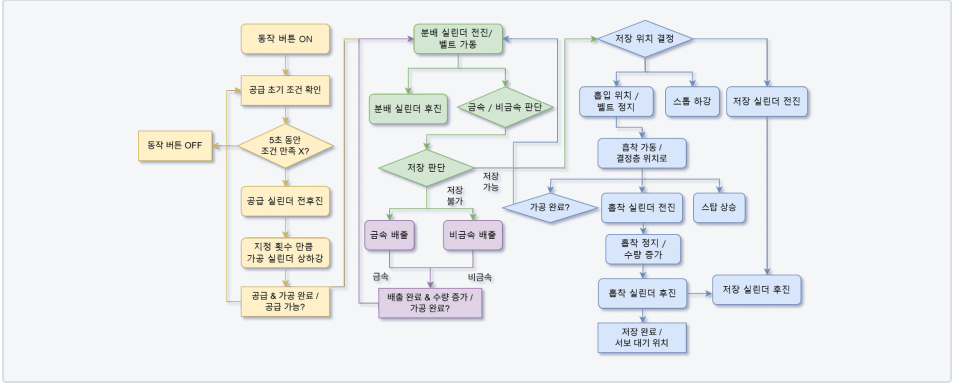


그림 8. 저장 판단과 적재 동작의 병렬 시퀀스

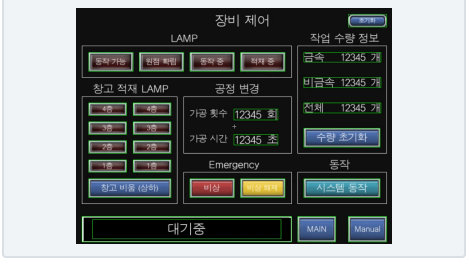


그림 9. 자동 운전·적재 상태



그림 10. 위치·속도 티칭

문제 1	병렬 운전 중 '빈 층을 찾았다'는 값과 '그 층에 소재가 있다'는 값이 한 메모리에 섞여 **이전 동작의 상태가 다음 시퀀스의 판단에 영향**
개선	**판단용·적재용 메모리 분리** 판단값은 적재가 끝나면 초기화하고, 적재값은 소재를 뺄 때까지 유지
문제 2	각자 작성한 코드를 통합하자 **같은 메모리 주소를 서로 다른 용도로 사용**해 시퀀스가 꼬임
개선	사용 중인 주소를 모두 모아 대조하고 **겹치는 주소 정리** 코멘트에 사용 주소를 적어 서로 확인
배운 점	코드 통합 전 **메모리 사용 주소 합의**가 중요함 병렬 동작에서는 장치별 동작 조건·완료 조건·인터록을 먼저 정해야 함